UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Deadlocks

Tarea 4.

Presenta:

Ramírez Juárez María Fernanda - 321204747

Rojo Peña Manuel Ianluck - 118005762

Profesor:

Javier León Cotonieto

Ayudante laboratorio:

Fernanda Garduño Ballesteros

Grupo: 7004, 2026-2

Fecha de entrega:

10 de marzo, 2026

Dealocks

Un **deadlock** ocurre cuando un conjunto de hilos queda bloqueado indefinidamente porque cada uno espera recursos que están retenidos por otro hilo del mismo conjunto.

En un sistema multiprogramado existen distintos **tipos de recursos** (CPU, archivos, dispositivos, locks), cada uno con una o más instancias.

El uso de un recurso sigue tres pasos:

1. **Solicitar el recurso**
2. **Utilizarlo**
3. **Liberarlo**

Si un hilo solicita un recurso ocupado, el sistema lo coloca en una cola de espera.

En aplicaciones multihilo, una causa común es adquirir **locks en orden diferente**. Por ejemplo, si un hilo adquiere L_1 y luego solicita L_2, mientras otro adquiere L_2 y solicita L_1, ambos quedan bloqueados permanentemente.

Livelock

El **livelock** es un fallo de vivacidad donde los hilos no están bloqueados pero tampoco progresan. Cada hilo detecta el conflicto y reintenta la operación, pero los reintentos simultáneos generan el mismo conflicto repetidamente.

Condiciones para Deadlock

Un deadlock solo puede ocurrir si se cumplen simultáneamente cuatro condiciones:

1. **Exclusión mutua**: al menos un recurso solo puede ser usado por un hilo a la vez.
2. **Retener y esperar**: un hilo mantiene recursos mientras espera otros adicionales.
3. **Sin apropiación**: los recursos no pueden ser retirados forzosamente, solo se liberan voluntariamente.
4. **Espera circular**: existe una cadena de hilos donde cada uno espera un recurso retenido por el siguiente.

Grafo de Asignación de Recursos

Los deadlocks pueden representarse mediante un grafo dirigido:

- Nodos de proceso (círculos)
- Nodos de recursos (rectángulos)

- Aristas de solicitud (proceso $\rightarrow$ recurso)
- Aristas de asignación (recurso $\rightarrow$ proceso)

Las propiedades del grafo son:

- Si no hay ciclos, no existe deadlock
- Si hay un ciclo y cada recurso tiene una sola instancia, el deadlock es seguro
- Con múltiples instancias, el ciclo solo indica posibilidad de deadlock

Métodos para Manejar Deadlocks

Los sistemas operativos pueden manejar los deadlocks mediante tres enfoques principales:

Ignorar el problema

Muchos sistemas operativos, como Linux y Windows, optan por ignorar los deadlocks cuando su ocurrencia es poco frecuente. En este enfoque la responsabilidad recae en los programadores, y si el sistema se bloquea normalmente se resuelve reiniciándolo.

Prevención o evitación

- **Prevención**: se diseña el sistema para que al menos una de las cuatro condiciones necesarias para el deadlock nunca se cumpla, restringiendo la forma en que se solicitan los recursos.
- **Evitación**: el sistema operativo analiza cada solicitud de recursos considerando el estado actual y las necesidades futuras de los procesos. Solo concede la solicitud si el sistema permanece en un estado seguro. El ejemplo más conocido es el **Algoritmo del Banquero**.

Detección y recuperación

Otra estrategia consiste en permitir deadlocks y detectarlos posteriormente. En este enfoque se permite que el deadlock ocurra, pero el sistema ejecuta periódicamente algoritmos que detectan ciclos de espera. Cuando se detecta uno, se aplican mecanismos de recuperación como abortar procesos o retirar recursos.

Detección:

Con una sola instancia por recurso se usa un **grafo de espera**, donde una arista $T_i \rightarrow T_j$ indica que T_i espera un recurso retenido por T_j. Un ciclo en el grafo implica deadlock.

Con múltiples instancias se utilizan estructuras similares al algoritmo del banquero y un algoritmo que identifica procesos que no pueden continuar.

Recuperación:

Una vez detectado el deadlock existen dos opciones:

- **terminación de procesos**: abortar uno o varios procesos para romper el ciclo
- **preempción de recursos**: retirar recursos a algunos procesos y reasignarlos

Prevención y Evitación

Los sistemas pueden manejar deadlocks mediante tres estrategias:

- **Ignorar el problema**: enfoque usado por muchos sistemas operativos cuando los deadlocks son poco frecuentes.
- **Prevención**: diseñar el sistema para romper al menos una de las cuatro condiciones necesarias.
- **Evitación**: permitir solicitudes de recursos solo si el sistema permanece en un estado seguro.

Prevención

La prevención rompe una de las condiciones necesarias:

- Eliminar la exclusión mutua cuando el recurso es compartible.
- Evitar retener y esperar solicitando todos los recursos al inicio.
- Permitir apropiación forzada de recursos.
- Imponer un orden total en la adquisición de recursos para evitar espera circular.

Estado Seguro y Grafo de Asignación

Estado Seguro

Un sistema está en **estado seguro** si existe una secuencia de hilos $\langle T_1, T_2, \ldots, T_n \rangle$ tal que cada hilo puede obtener los recursos que necesita usando los recursos disponibles más los liberados por los hilos anteriores. A esta secuencia se le llama **secuencia segura**.

Un estado seguro garantiza que no habrá deadlock. En cambio, un estado **inseguro** no implica necesariamente un deadlock, pero el sistema ya no puede garantizar que este no ocurra.

En los algoritmos de evitación, una solicitud de recursos se concede solo si la asignación mantiene al sistema en un estado seguro.

Algoritmo del Grafo de Asignación

Cuando cada tipo de recurso tiene una sola instancia, el grafo de asignación puede ampliarse con **aristas de reclamación**, que indican que un hilo podría solicitar un recurso en el futuro.

Cuando un hilo solicita un recurso, el sistema simula la asignación y verifica si el grafo resultante contiene ciclos. Si no se forma ningún ciclo, la solicitud se concede; de lo contrario, el hilo debe esperar.

Este método tiene un costo aproximado de $O(n^2)$ y es aplicable solo cuando cada recurso tiene una única instancia. Para múltiples instancias se utiliza el **Algoritmo del Banquero**.

Algoritmo del Banquero

El algoritmo del banquero evita deadlocks verificando que cada asignación mantenga al sistema en un **estado seguro**.

Un estado es seguro si existe una secuencia de procesos tal que cada uno pueda terminar usando los recursos disponibles más los liberados por los anteriores.

El algoritmo mantiene cuatro estructuras:

- `Disponible`: recursos libres
- `Maximo`: demanda máxima de cada proceso
- `Asignado`: recursos actualmente asignados
- `Necesita` = Maximo − Asignado

Antes de conceder una solicitud, el sistema simula la asignación y ejecuta un algoritmo de seguridad. Si el estado resultante es seguro, la solicitud se concede; de lo contrario, el proceso debe esperar.

Ejercicio 8.28

Considere la siguiente *snapshot* de un sistema:

	Allocation				***Max***				***Available***			
	A	B	C	D	A	B	C	D	A	B	C	D
T_0	3	1	4	1	6	4	7	3	2	2	2	4
T_1	2	1	0	2	4	2	3	2				
T_2	2	4	1	3	2	5	3	3				
T_3	4	1	1	0	6	3	3	2				
T_4	2	2	2	1	5	6	7	5				

Responda las siguientes preguntas usando el ***Algoritmo del Banquero***:

a. Ilustre que el sistema está en un estado seguro demostrando un orden en el cual los hilos pueden completar su ejecución.

b. Si llega una solicitud del hilo T_4 para $(2, 2, 2, 4)$, ¿puede concederse inmediatamente?

c. Si llega una solicitud del hilo T_2 para $(0, 1, 1, 0)$, ¿puede concederse inmediatamente?

d. Si llega una solicitud del hilo T_3 para $(2, 2, 1, 2)$, ¿puede concederse inmediatamente?

Calculamos la matriz `Need` se calcula como:

$$\texttt{Need} = \texttt{Max} - \texttt{Allocation}$$

Hilo	A	B	C	D
T_0	3	3	3	2
T_1	2	1	3	0
T_2	0	1	2	0
T_3	2	2	2	2
T_4	3	4	5	4

a) Verificar si el sistema está en estado seguro

Aplicamos el algoritmo de seguridad.

$$\texttt{Work} = \texttt{Available} = \texttt{(2,2,2,4)}$$

1° Buscamos el hilo con $\texttt{Need} \leq \texttt{Work}$:

T_2 lo cumple pues: $\texttt{Need} = (0, 1, 2, 0) \leq (2, 2, 2, 4)$

T_2 puede correr:

$$\texttt{Work} = (2, 2, 2, 4) + (2, 4, 1, 3) = (4, 6, 3, 7)$$

2° T_0: $\texttt{Need} = (3,3,3,4) \leq (4,6,3,7)$

T_0 puede correr:

$$\texttt{Work} = (4,6,3,7) + (3,1,4,1) = (7,7,7,8)$$

3° T_1: $\texttt{Need} = (2,1,3,0) \leq (7,7,7,8)$

T_1 puede correr:

$$\texttt{Work} = (7,7,7,8) + (2,1,0,2) = (9,8,7,10)$$

4° T_3: $\texttt{Need} = (2,2,2,2) \leq (9,8,7,10)$

T_3 puede correr:

$$\texttt{Work} = (9,8,7,10) + (4,1,1,0) = (13,9,8,10)$$

5° T_4: $\texttt{Need} = (3,4,5,4) \leq (13,9,8,10)$

Por lo tanto existe la secuencia segura:

$$\langle T_2, T_0, T_1, T_3, T_4 \rangle$$

$$\boxed{\text{El sistema está en } \textbf{estado seguro}.}$$

b) Solicitud de $T_4 = (2,2,2,4)$

1. Verificación de límite:

$$(2,2,2,4) \leq \texttt{Need}_{T_4} = (3,4,5,4)\checkmark$$

2. Verificación de disponibilidad:

$$(2,2,2,4) \leq \mathit{Available} = (2,2,2,4)\checkmark$$

3. Simulación:

$$\texttt{Available} = (0,0,0,0)$$

Con $\texttt{Work} = (0,0,0,0)$ ningún hilo puede avanzar, por lo que ningún proceso puede satisfacer su necesidad ($\texttt{Need}_i > 0$ para todos los procesos).

Por lo tanto el sistema quedaría en un **estado inseguro**.

$$\boxed{\text{La solicitud } T_4 \text{ no puede consederse y se deniega.}}$$

c) Solicitud de $T_2 = (0, 1, 1, 0)$

Verificaciones:

$$(0, 1, 1, 0) \leq \texttt{Need}_{T_2} = (0, 1, 2, 0)$$

$$(0, 1, 1, 0) \leq \texttt{Available} = (2, 2, 2, 4)$$

Simulación:

$$\texttt{Available} = (2, 1, 1, 4)$$

$$\texttt{Need}_{T_2} = (0, 1, 2, 0) - (0, 1, 1, 0) = (0, 0, 1, 0)$$

Verficando seguridad con $\texttt{Work} = (2, 1, 1, 4)$:

Turno	**Hilo elegido**	**Need**	**$\leq$ Work?**	**Work resultante**
1	T_2	$(0, 0, 1, 0)$	✓	$(4, 6, 3, 7)$
2	T_0	$(3, 3, 3, 2)$	✓	$(7, 7, 7, 8)$
3	T_1	$(2, 1, 3, 0)$	✓	$(9, 8, 7, 10)$
4	T_3	$(2, 2, 2, 2)$	✓	$(13, 9, 8, 10)$
5	T_4	$(3, 4, 5, 4)$	✓	*None*

Se obtiene la secuencia:

$$\langle T_2, T_0, T_1, T_3, T_4 \rangle$$

Por lo tanto el estado sigue siendo seguro.

$$\boxed{\text{La solicitud } T_2 \text{ sí puede concederse.}}$$

d) Solicitud de $T_3 = (2, 2, 1, 2)$

Verificaciones:

$$(2, 2, 1, 2) \leq \texttt{Need}_{T_3} = (2, 2, 2, 2)$$

$$(2, 2, 1, 2) \leq \texttt{Available} = (2, 2, 2, 4)$$

Simulación:

$$\texttt{Available} = (2, 2, 2, 4) - (2, 2, 1, 2) = (0, 0, 1, 2)$$
$$\texttt{Allocation}_{T_3} = (4, 1, 1, 0) + (2, 2, 1, 2) = (6, 3, 2, 2)$$
$$\texttt{Need}_{T_3} = (2, 2, 2, 2) - (2, 2, 1, 2) = (0, 0, 1, 0)$$

Verrificamos seguridad con $\texttt{Work} = (0, 0, 1, 2)$:

Turno	**Hilo elegido**	**Need**	$\leq$ **Work?**	**Work resultante**
1	T_3	$(0, 0, 1, 0)$	✓	$(6, 3, 3, 4)$
2	T_0	$(3, 3, 3, 2)$	✓	$(9, 4, 7, 5)$
3	T_1	$(2, 1, 3, 0)$	✓	$(11, 5, 7, 7)$
4	T_2	$(0, 1, 2, 0)$	✓	$(13, 9, 8, 10)$
5	T_4	$(3, 4, 5, 4)$	✓	*None*

Se obtiene la secuencia:

$$\langle T_3, T_0, T_1, T_2, T_4 \rangle$$

El sistema permanece en estado seguro. Y por lo tanto

$$\boxed{\text{La solicitud } T_3 \text{ sí puede concederse.}}$$

Resultados

Inciso	Solicitud	Resultado
a	Estado inicial	Seguro
b	$T_4 : (2, 2, 2, 4)$	Denegada
c	$T_2 : (0, 1, 1, 0)$	Concedida
d	$T_3 : (2, 2, 1, 2)$	Concedida